

# Flyweight

## 1. Problema

## 2. Solución

## 3. Participantes

## 4. Diagrama UML

## 5. Código

5.1. Clase `Rol` (Flyweight)

5.2. Clase `RolFactory` (FlyweightFactory)

5.3. Clase `Usuario` (Contexto)

5.4. Uso del código

## 6. Conclusión

## 1. Problema

En la aplicación de registro de usuarios para un sistema médico, se detectó que ciertos valores como el **rol** del usuario ( `cliente` , `médico` , `admin` , etc.) eran **repetitivos** entre múltiples instancias de objetos `Usuario` . Esto genera un **consumo innecesario de memoria**, especialmente si la cantidad de usuarios aumenta considerablemente.

---

## 2. Solución

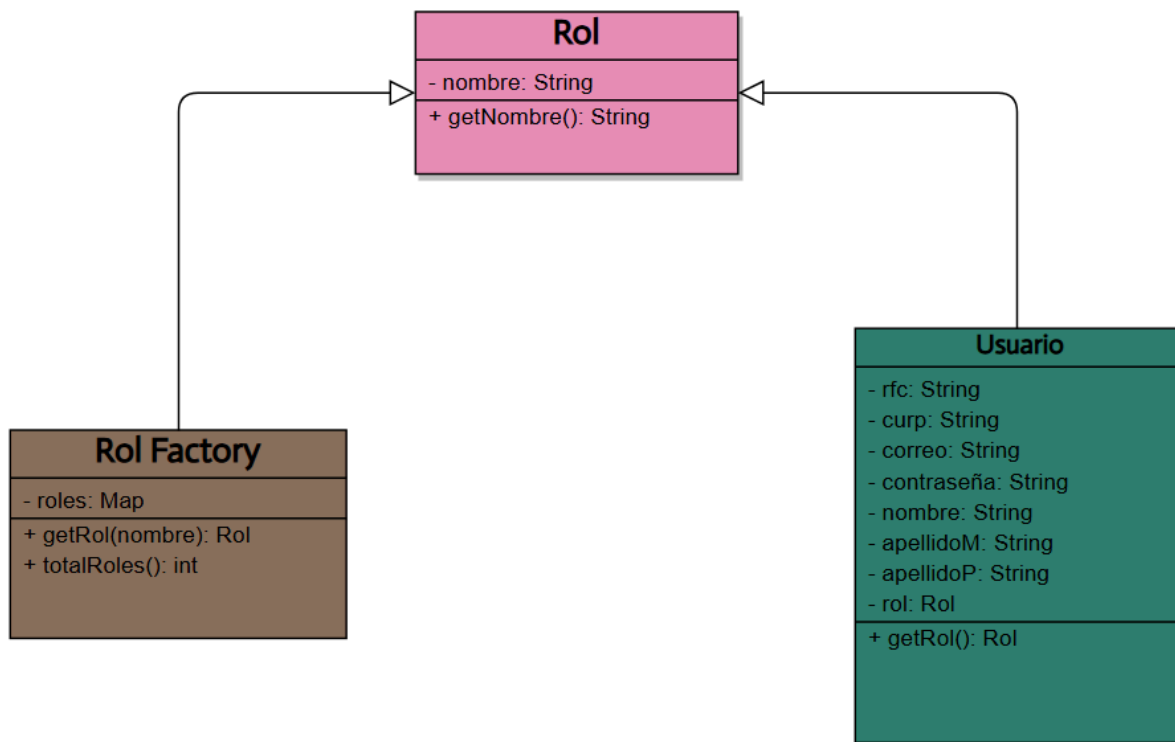
Para optimizar el uso de memoria, se implementó el **patrón de diseño Flyweight**. Este patrón permite **compartir objetos inmutables** que son comunes a muchas instancias. En este caso, se decidió compartir el objeto `Rol` , encapsulando el nombre del rol en una clase dedicada y utilizando una fábrica ( `RolFactory` ) que se encarga de reutilizar las instancias ya existentes.

---

## 3. Participantes

- **Flyweight (Rol):** Clase que representa el objeto compartido. Contiene únicamente el nombre del rol.
- **Flyweight Factory (RolFactory):** Se encarga de gestionar las instancias únicas de cada tipo de rol.
- **Context (Usuario):** Clase que contiene una referencia al Flyweight ( Rol ) junto con otros datos específicos del usuario.
- **Cliente:** La clase que registra nuevos usuarios y obtiene los objetos Rol desde la fábrica.

## 4. Diagrama UML



## 5. Código

El código sigue la estructura del **Patrón Flyweight**.

### 5.1. Clase **Rol** (Flyweight)

```
/**
 *
 * @author kiwik
 */

public class Rol {
    private final String nombre;

    public Rol(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }
}
```

### 5.2. Clase **RolFactory** (FlyweightFactory)

Esta clase recibirá los eventos de cambio y realizará las acciones necesarias.

```

package Clases;

import java.util.HashMap;
import java.util.Map;

public class RolFactory {
    private static final Map<String, Rol> roles = new HashMap<>();

    public static Rol getRol(String nombre) {
        Rol rol = roles.get(nombre.toLowerCase());

        if (rol == null) {
            rol = new Rol(nombre);
            roles.put(nombre.toLowerCase(), rol);
        }

        return rol;
    }

    public static int totalRoles() {
        return roles.size();
    }
}

```

### 5.3. Clase **Usuario** (Contexto)

Proporciona el objeto usuario ya modificado con el rol necesitado.

```

public class Usuario {
    private String rfc, curp, correo, contraseña, nombre, apellidoMaterno, apellidoPaterno, tipo;
    private int id;
    private Rol rol;
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }

    public Usuario(String rfc, String curp, String correo, String contraseña, String nombre, String apellidoMaterno, String apellidoPaterno, Rol rol) {
        this.rfc = rfc;
        this.curp = curp;
        this.correo = correo;
        this.contraseña = contraseña;
        this.nombre = nombre;
        this.apellidoMaterno = apellidoMaterno;
        this.apellidoPaterno = apellidoPaterno;
        this.rol = rol;
    }

    public Rol getRol() {
        return rol;
    }

    public String getTipo() {
        return tipo;
    }

    public String getRfc() {
        return rfc;
    }
}

```

## 5.4. Uso del código

Modificamos todas las partes al momento de crear un nuevo usuario y los declaramos de la siguiente forma.

```

// Crear el objeto Usuario si la contraseña coincide
Rol rolUsuario = RolFactory.getRol("cliente");
Usuario usuario = new Usuario(rfcText, curpText, correoText, contraseñaText,
    Nombre, apellidoMaternoText, apellidoPaternoText, rolUsuario);

try {
    if (RolFactory.getRol(rolUsuario) != null) {

```

## 6. Conclusión

La implementación del patrón Flyweight en este proyecto permitió optimizar el uso de memoria al reutilizar instancias de roles compartidos entre múltiples usuarios. Al separar el estado compartido (los roles) del estado único (los datos del usuario), se logró una mayor eficiencia y un diseño más limpio y escalable. Este patrón es especialmente útil en sistemas donde se manejan muchos objetos similares, como en sistemas de autenticación o gestión de usuarios.